给 AI 发通行证：构建 Agent 身份与权限系统

题目：给 AI 发通行证：构建 Agent 身份与权限系统

课题描述

本课题聚焦多 Agent 协作场景下身份混淆、权限滥用、信任链缺失等安全痛点，打造面向 AI Agent 的通用身份与权限系统。参赛同学需完成 Agent 身份认证、细粒度权限管控、委托授权、审计追溯等核心模块开发，交付可运行的多 Agent 协作安全原型系统，实现多 Agent 协作全流程安全可控、可追溯，为 AI Agent 企业级落地筑牢安全底座。

快速理解

> 想象飞书里有三个 AI 助手：飞书文档助手 Agent 负责生成飞书报告；企业数据 Agent 是唯一有权访问飞书通讯录、知识库、多维表格的 AI；外部检索 Agent 只能抓取公开网页信息。
>
> 当「文档助手 Agent」想委托「企业数据 Agent」读取数据时，系统怎么确认这个委托合法？当其他非法 Agent 让「企业数据 Agent」查询企业数据，怎么拦截？
>
> 你的任务：给每个 AI 发一张专属「通行证」，定义它能做什么、不能做什么，并记录每一次操作的完整日志。这就是「给 AI 的 IAM 系统」。

关键术语速查（不懂安全背景也没关系，直接开始做）：

术语	白话解释
Access Token	每个 AI 的「通行证」，用来证明「有权限做某个事情」
Capability（能力声明）	通行证上标注的「只能做哪些事」，如「只能读文档，不能删数据库」
Delegated Authorization（委托授权）	Agent A 代表用户 User 去做事
Chain of Trust（信任链）	谁信任谁的可验证证明链，类似逐层盖章背书
JWT（JSON Web Token）	一种通用 Token 格式标准
静态授权/动态授权	静态授权：Agent 注册时由管理员预先固化在系统中的权限集合，运行期间不变。 动态授权：运行时根据上下文临时计算并颁发的权限，Token 到期自动失效。

背景

传统的身份认证与权限体系（IAM）是围绕“人”建立的，其基本模型是 `User → Application`。但在 AI 时代，工作流正在演变为一条更长、更复杂的调用链：`User → Agent A → Agent B → Service C`。

在这条新范式下，原有的安全假设被打破，一系列严峻的痛点浮出水面：

1. **身份混淆与权限滥用**：当 Agent B 收到 Agent A 的请求时，它没法确定这个请求到底是 Agent A 发的，还是有“人”或者其他“Agent”冒充 Agent A 发的。要是 Agent B 本身权限很高，又不做任何身份验证，就可能会被越权调用，进而触发破坏性任务或者导致数据泄漏。
2. **信任链的缺失**：在多 Agent 协作中，信任无法通过网络边界或简单的 API Key 来传递。Agent C 如何信任 Agent B，Agent B 如何信任 Agent A？如果不能建立一条可验证的 **信任链 (Chain of Trust)**，整条协作链路将充满不确定性，任何一个环节的失陷都可能导致整个系统的崩溃或数据泄露。
3. **审计与追溯的黑洞**：一旦发生安全事件，我们如何追溯一个决策是由哪个 Agent、在哪条调用链的哪个环节、代表哪个用户做出的？没有一套清晰的 **Agent 间调用（A2A Auth）** 审计日志，安全事件的响应和定责将成为不可能完成的任务。

为 AI Agent 建立一套专属的身份与权限基础设施，不再是锦上添花，而是决定其能否在企业核心场景中安全落地的“生死线”。

能力考查重点

- **协议抽象与系统设计能力**：你能否将复杂的授权逻辑抽象为通用的 **A2A (Agent-to-Agent) 认证与授权协议**，并设计出类似 **OAuth 2.0** 的标准化令牌（Token）与交互流程？
- **多 Agent 编排能力**：你对 **Agent Identity (智能体身份)**、**Delegation (委托授权)** 和 **Capability-Based Authorization (基于能力的授权)** 等核心概念的理解有多深？
- **工程实现能力**：你是否能将你的架构设计转化为可运行、可演示、高可用的原型系统？
- **可观测与可验收能力**：是否具备清晰的授权审计日志与追溯能力？

具体命题

你需要构建一个 **Agent 身份与权限系统**，该服务能够：

- **签发与验证 Access Token**：为每个 Agent 建立唯一、可校验的身份，并支持在调用链中验证请求来源。
- **管理能力声明 (Capability Statement)**：定义不同 Agent 或不同角色的 Agent 所拥有的细粒度操作权限。
- **处理委托授权 (Delegated Authorization)**：当 Agent 代表真实用户执行操作时，能够正确计算 `用户权限 ∩ Agent能力` 的有效权限交集。

- **实时拦截越权行为**：在 Agent 调用另一个 Agent 时，强制进行权限校验，并明确拒绝任何超出其授权范围的请求。
- **提供审计可追溯性**：记录每一次授权决策（允许或拒绝）的完整上下文，形成审计日志。

推荐演示场景（飞书企业协作场景）：

推荐使用以下飞书业务结合场景，以飞书 OpenAPI 作为 Agent 工具调用的目标资源：

- **飞书文档助手 Agent**：负责理解用户需求，调用其他Agent完成企业数据查询和外部信息检索，并将最终报告写入飞书文档。
- **企业数据 Agent**：唯一有权通过飞书 OpenAPI 访问飞书通讯录、日历、多维表格数据的 Agent。
- **外部检索 Agent**：负责从外部公开网站获取信息，**无权访问任何飞书企业内部数据**。

你需要演示：

1. **正常委托流程**：「飞书文档助手」为生成报告，成功委托「企业数据 Agent」读取多维表格数据，最终将报告写入飞书文档，全程授权合法并生成审计日志。
2. **越权拦截**：「外部检索 Agent」尝试委托「企业数据 Agent」读取多维表格数据，「外部检索 Agent」因缺乏对应的权限，请求被框架**成功拦截**，返回明确错误码，并在审计日志中记录拦截信息。

任务目标与范围

必做范围：

- 设计一套清晰的 Agent 协作协议，至少包含以下对象或概念：
  - **Agent 身份与角色定义**：通过 `Access Token` 中的 `sub` 或 `agent_id` 声明实现。
  - **任务描述与任务状态**：本方案中体现为 Agent 执行具体操作的意图，如“访问数据库”。
  - **子任务拆解与任务依赖关系**：本方案中体现为 Agent 间的调用链，如“报告 Agent”依赖“数据 Agent”。
  - **上下文传递与引用机制**：通过 `Access Token` 中的 `delegated_user` 传递用户上下文。
  - **结果回传与结果聚合**：本方案中体现为被调用方返回数据或拒绝访问的响应。
  - **异常、失败、求助或升级处理机制**：通过明确的、语义化的错误信息（如能力不足、Token 过期）来响应失败。
- 实现一个可运行的多 Agent 协作原型系统。
  - 你需要交付一个 **Agent 身份与权限系统**，它必须能够支撑起一个典型的 `User → Agent A → Agent B` 调用链。
- 至少接入 3 个具备不同职责的 Agent。
  - 如“具体命题”中建议的：**飞书文档助手 Agent**、**企业数据 Agent**、**网络检索 Agent**。
- 系统需具备任务链路可视化或可追踪能力，能够看到任务如何在多个 Agent 间流转。

- 通过提供**可查询、可导出的审计日志**来实现，清晰展示每一次授权决策（无论是允许还是拒绝）的完整上下文。

不要求范围：

- 不要求做成完整商业化平台。
- 不要求覆盖所有 Agent 框架。
- 不要求追求超大规模分布式部署，但架构上需要具备扩展性说明。

演示与验收要求

评分维度：

评分维度	最低达标要求（必须通过）	优秀标准（加分）
协议与架构设计	• 有架构图 • Access Token 字段说明 • 审计日志 字段说明	协议文档完整，覆盖异常与边界流程
功能实现	• 3个 Agent 可完成一次正常协作 • 实现身份认证 • 实现 Agent 静态授权 • 具备审计与追溯	• 实现 Agent 动态授权 • 针对 Agent 行为不可预测性有具体设计（如Token 实时撤销机制） • 能够对抗Access Token盗用 • 支持异构 Agent 接入，例如不同模型、不同推理引擎、不同工具集
工程质量	• 系统无明显bug • 有 README + 启动脚本，评委可独立运行	具备系统异常感知能力，譬如监控告警等
系统易用性	• 交互体验流畅，支持自然语言指令驱动全链路任务执行	• 错误提示清晰友好，新手无需阅读文档即可上手 • 提供可视化界面或便捷 CLI，显著降低操作门槛
领域认知	• 能说明为何传统 IAM 无法直接套用于 AI Agent 场景 • 理解 Agent 身份模型与人类用户身份的区别	• 知道 Prompt Injection 等 AI 特有攻击对权限系统的威胁 • 能分析主流 Agent 框架在 IAM 层面的不足并提出改进思路

标准验收步骤（评委将按此顺序独立复现）：

1. **启动系统**：运行启动脚本，所有 Agent 和授权服务正常启动，无报错

2. **正常委托流程**：触发「飞书文档助手」→「企业数据 Agent」委托，观察数据返回正常
3. **越权拦截流程**：触发「外部检索 Agent」→ 「企业数据 Agent」委托，尝试访问飞书企业数据，观察请求被拦截，返回错误码
4. **日志完整性验收**：查看审计日志，确认以上两条记录

演示要求：

- 展示系统架构、协议设计、Access Token/审计日志核心数据结构
- 明确展示多个 Agent 的职责分工、交互链路和结果汇聚过程
- 展示异常场景处理：某个 Agent 超时、失败或权限不足时，系统如何响应和兜底

验收要求：

- 提交项目说明文档，包含：技术方案设计文档（架构、Access Token 设计、A2A 认证流程、API 接口定义）、安装与运行指南、演示脚本（每步操作指令 + 预期结果）
- 提交可运行代码或可部署原型
- 提供演示视频或现场演示环境
- 评委可基于参赛者给出的步骤完成基础复现

直接不通过红线（满足任一条即淘汰）：

- 核心安全逻辑（Token 签发、验证、拦截）无真实代码实现，仅界面 Mock 或伪造日志